

CPU Scheduling

1. Why Scheduling Exists

In a multiprogramming system, many processes sit in the **ready queue** at once, but (on a single-core CPU) only one can actually run at a time. **CPU scheduling** is the policy the OS uses to decide *which* ready process gets the CPU next.

Recall from Process Management: scheduling happens at three levels —

Scheduler	Moves process	Frequency
Long-term	New → Ready	Low (controls degree of multiprogramming)
Mid-term	Blocked ↔ Ready (via suspend states)	Medium
Short-term (CPU scheduler)	Ready → Running	High (this is the "CPU scheduling" this topic is about)

The short-term scheduler runs constantly — every time the CPU becomes free, it must pick the next process. This is the frequency that matters most for performance, which is why CPU scheduling *algorithms* are a big topic on their own.

2. Scheduling Criteria

An algorithm is judged on:

- **Maximize CPU utilization** — keep the CPU as busy as possible.
- **Maximize throughput** — number of processes completed per unit time.
- **Minimize waiting time** — total time a process spends sitting in the ready queue (not running, not blocked — just waiting for the CPU).
- **Minimize turnaround time** — total time from arrival to completion.

Turnaround time = Waiting time + Execution (burst) time
- **Minimize response time** — time from arrival until the process gets its *first* response from the CPU (important for interactive systems; distinct from turnaround time, which cares about *completion*, not first response).

No algorithm can be optimal on all five simultaneously — every scheduling algorithm is a different trade-off among these.

3. First Come First Served (FCFS)

- **Non-preemptive.** The process that arrives first is serviced first, in strict arrival order.
- Implemented as a plain queue: process joins the back, leaves from the front once it completes.

Example:

Process	Arrival Time	Execution Time
P1	0	9
P2	0	10
P3	0	5
P4	0	3
P5	0	7

Gantt chart:

```
| P1 | P2 | P3 | P4 | P5 |  
0   9   19  24  27  34
```

Waiting time = start time of each process (since all arrive at 0):

P1=0, P2=9, P3=19, P4=24, P5=27 → **Avg WT = $79/5 = 15.8$**

Turnaround time = Waiting time + Execution time:

P1=9, P2=19, P3=24, P4=27, P5=34 → **Avg TAT = $113/5 = 22.6$**

Advantage

- Simple, and **unbiased** — nobody can be favored by tricks like gaming a priority value; order is purely arrival-based.

Disadvantage — the Convoy Effect

When a **short** process gets stuck in the queue behind a **long** process, its waiting time balloons — even though it needed almost no CPU time itself.

Example: P1 (burst 24) arrives before P2 (burst 3).

- If order is P1 then P2: WT(P1)=0, WT(P2)=24 → **Avg WT = 12**

- If order is P2 then P1: WT(P1)=3, WT(P2)=0 → **Avg WT = 1.5**

Same two processes, wildly different average waiting time — purely because of arrival order. This "small process trapped behind a big one" scenario is the **convoy effect**, and it's FCFS's single biggest weakness.

4. Shortest Job First (SJF)

Two versions exist:

- **Non-preemptive SJF** — once a process starts, it runs to completion; among ready processes, always pick the one with the shortest **total** execution time.
- **Preemptive SJF**, also called **SRTF (Shortest Remaining Time First)** — at every arrival, compare the new process's burst time against the *remaining* time of the currently running process; preempt if the new one is shorter. Involves more context switches than non-preemptive SJF.

Example (non-preemptive, all arrive at 0):

Process	Execution Time
P1	9
P2	10
P3	5
P4	3
P5	7

Gantt chart:

```
| P4 | P3 | P5 | P1 | P2 |  
0   3   8  15  24  34
```

Advantages

- **Provably minimizes average waiting time** among non-preemptive algorithms, for a given set of arrival times.
- Maximizes throughput.

Disadvantages

- **Starvation risk** — a long process can be perpetually skipped if shorter processes keep arriving.
- Requires knowing burst time in advance — in practice this must be *estimated* (e.g., via exponential averaging of past bursts), since the OS can't truly know the future.
- More context switches in the preemptive (SRTF) version compared to FCFS.

5. Priority Scheduling

- Each process has a **priority** value (lower value = higher priority is the common convention, though this can be reversed by convention).
- Comes in both **preemptive** and **non-preemptive** flavors.

- **Preemptive:** if a newly-arrived process has higher priority than the currently running one, it immediately preempts it.

Example (priority: lower number = higher priority):

Process	AT	ET	Priority
P1	0	9	3
P2	0	10	1
P3	0	5	0
P4	0	3	4
P5	0	7	2

Non-preemptive order (by priority): P3, P2, P5, P1, P4.

Advantages

- Lets the system express *importance*, not just size — a critical process can jump the queue.

Disadvantages

- **Starvation** — a low-priority process may never run if higher-priority processes keep arriving.
- **Fix: Aging** — gradually increase the priority of a process the longer it waits, guaranteeing it eventually becomes the highest priority and runs.

6. Round Robin (RR)

- **Pure preemptive.** Every process gets the CPU for a fixed slice of time — the **time quantum (time slice)** — then, if not finished, is preempted and sent to the back of the ready queue.
- Explicitly designed for time-shared, interactive systems — its objective is to **minimize response time**, not necessarily average waiting/turnaround time.

Behavior at the extremes of time quantum size:

- **As $tq \rightarrow 0$:** approaches the (unrealistic) ideal where every process seems to have its own dedicated (slow) processor — but in practice, context-switch overhead would dominate completely, making this a poor choice.
- **As $tq \rightarrow \infty$:** RR degenerates into **FCFS**, since no process is ever preempted before finishing.

Example (Time Quantum = 3, all arrive at 0):

Process	ET
P1	9
P2	10
P3	5
P4	3
P5	7

	P1		P2		P3		P4		P5		P1		P2		P3		P5		P1		P2		P5	
0		3		6		9		12		15		18		21		23		26		29		32		34

Advantage

- Low response time — great for interactive/time-shared systems.

Disadvantage

- **Choosing the time quantum is a balancing act.** Too small → excessive context-switch overhead. Too large → behaves like FCFS and reintroduces the convoy effect.
- Higher average waiting/turnaround time than SJF in general, since it deliberately sacrifices those for fairness/responsiveness.

7. Multilevel Queue (MLQ) Scheduling

- The ready queue is split into **multiple separate queues**, based on some process classification (e.g., system processes, interactive processes, batch processes).
- Each queue can have its **own scheduling algorithm** (e.g., RR for interactive, FCFS for batch).
- Queues themselves are typically prioritized relative to each other (e.g., strict priority between queues, or a percentage split of CPU time — e.g., 60% high-priority queue, 30% medium, 10% low).

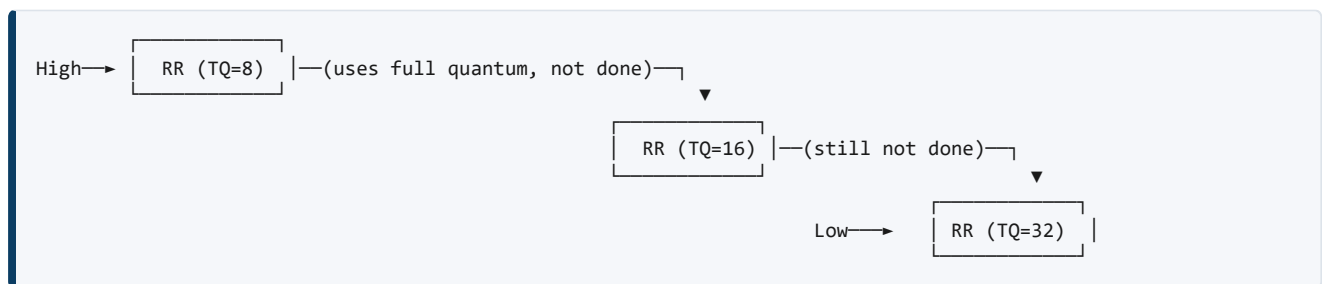
Process → classified by priority/type → placed into one queue		
High-priority		60%
Medium-priority		30%
Low-priority		10%

Key limitation

- A process is **permanently assigned** to a queue when created — it cannot move between queues. A process misclassified early (or one whose behavior changes over time) is stuck with a potentially unfair scheduling policy forever.

8. Multilevel Feedback Queue (MLFQ) Scheduling

- Like MLQ, but processes **can move between queues** based on their observed behavior — this is what "feedback" means.
- **All processes enter the highest-priority (topmost) queue first.**
- A common rule: if a process uses its *entire* time quantum in a queue without finishing (behaving CPU-bound), it's demoted to a lower-priority queue with a larger time quantum. If it *voluntarily* gives up the CPU early (e.g., blocking for I/O — behaving I/O-bound/interactive), it stays at (or is promoted back to) a higher-priority queue.



Why this is better than MLQ

- **Self-correcting:** the scheduler doesn't need to know in advance whether a process is CPU-bound or I/O-bound — it *infers* it from behavior and adjusts. This makes MLFQ one of the most general and widely-used scheduling frameworks in real operating systems.

Trade-off

- Considerably more complex to implement and tune (how many queues, what quantum per queue, promotion/demotion rules) than any single algorithm above.
-

9. Summary Comparison

Algorithm	Preemptive?	Best for	Main weakness
FCFS	No	Simplicity, fairness in arrival order	Convoy effect
SJF (non-preemptive)	No	Minimizing avg. waiting time	Starvation of long jobs; needs burst-time estimate
SRTF (preemptive SJF)	Yes	Minimizing avg. waiting time (dynamic)	More context switches; starvation
Priority	Either	Expressing importance	Starvation (fixed by aging)
Round Robin	Yes	Interactive/time-shared systems	Time-quantum tuning; degrades to FCFS or CS-overhead extremes
MLQ	Depends per queue	Distinct, unchanging process classes	No movement between queues
MLFQ	Yes	General-purpose systems where process behavior varies/is unknown	Implementation/tuning complexity

Interview soundbite: "Every CPU scheduling algorithm is a trade-off along the same five axes — utilization, throughput, waiting time, turnaround time, and response time. FCFS is simple but suffers the convoy effect. SJF/SRTF is provably optimal for average waiting time but needs burst-time knowledge and risks starving long jobs. Priority scheduling expresses importance but also risks starvation, fixed by aging. Round Robin optimizes for response time via time-slicing, at the cost of tuning the time quantum correctly. MLFQ is the most practical general-purpose approach because it infers whether a process is CPU-bound or I/O-bound from its behavior and adjusts automatically, without needing to know that in advance."

Interview Questions With Answers

Q1. What are the five criteria used to evaluate a CPU scheduling algorithm?

Answer: CPU utilization (maximize), throughput (maximize), waiting time (minimize), turnaround time (minimize), and response time (minimize). No single algorithm optimizes all five at once — each algorithm makes a different trade-off among them.

Q2. What is turnaround time, and how is it calculated?

Answer: Turnaround time is the total time from a process's arrival until its completion. It equals waiting time plus execution (burst) time: $TAT = WT + ET$.

Q3. What is the convoy effect, and which algorithm suffers from it most?

Answer: The convoy effect happens when a short process gets stuck waiting behind a long process in the queue, dramatically increasing its (and the average) waiting time even though it needed very little CPU time. FCFS suffers from this most directly, since it has no mechanism to let a short process "cut in line" ahead of a long one that arrived earlier.

Q4. Why is SJF considered optimal, and what stops us from using it in practice?

Answer: SJF (non-preemptive) provably minimizes average waiting time for a given set of processes and arrival times, because running shorter jobs first means fewer processes accumulate behind any one job. In practice, we can't use "true" SJF because the OS doesn't know a process's exact future burst time in advance — it can only estimate it (e.g., using an exponentially weighted average of that process's past CPU bursts), and a bad estimate undermines the whole benefit.

Q5. What's the difference between SJF and SRTF?

Answer: SJF (non-preemptive) picks the shortest job among ready processes and runs it to completion once started. SRTF (Shortest Remaining Time First) is the preemptive version — every time a new process arrives, it compares that new process's burst time against the *remaining* execution time of whatever is currently running, and preempts if the new one is shorter. SRTF generally causes more context switches.

Q6. How does aging solve starvation in priority scheduling?

Answer: Aging gradually increases a waiting process's priority the longer it stays in the ready queue. Even a process that started at the lowest priority will eventually have its priority raised high enough to be scheduled, guaranteeing it can't be starved forever by a continuous stream of higher-priority arrivals.

Q7. What happens to Round Robin as the time quantum approaches infinity? As it approaches zero?

Answer: As the time quantum $\rightarrow \infty$, no process is ever preempted before finishing, so RR degenerates into FCFS (with all its convoy-effect problems). As the time quantum $\rightarrow 0$, in theory every process seems to get its own slow dedicated processor (extremely fair, extremely low apparent per-process wait before *some* progress), but in practice the overhead of extremely frequent context switching would dominate and destroy actual throughput — so this extreme is impractical, not actually desirable.

Q8. What's the core difference between Multilevel Queue and Multilevel Feedback Queue scheduling?

Answer: In MLQ, a process is permanently assigned to one queue at creation and can never move to another — it's stuck there even if its behavior would be better served elsewhere. In MLFQ, processes can move between queues based on their observed behavior (e.g., getting demoted to a lower-priority, larger-quantum queue if they behave CPU-bound, or staying/promoting to a higher-priority queue if they behave I/O-bound/interactive) — this "feedback" is exactly what the name refers to.

Q9. Why is MLFQ described as "self-correcting" or able to infer process behavior?

Answer: Because it doesn't need to know in advance whether a process is CPU-bound or I/O-bound (which real schedulers can't know for certain ahead of time). Instead, it watches what the process actually does: if a process consistently uses its entire time quantum without voluntarily giving up the CPU, it's treated as more CPU-bound and demoted to a lower-priority queue; if it gives up the CPU early (e.g., blocking on I/O), it's treated as more interactive and kept at/promoted to a higher-priority queue. The scheduler adapts based on evidence rather than upfront assumptions.

Q10. Why doesn't Round Robin minimize average waiting time as well as SJF, even though it's fairer?

Answer: RR is optimized for a different goal — minimizing response time (fast first feedback to every process) — not minimizing average waiting time. Because it deliberately spreads CPU time evenly across all ready processes via time-slicing, longer processes end up delaying shorter ones across many rounds instead of letting short processes finish quickly and get out of the way, which is exactly what SJF optimizes for. Fairness (RR's goal) and minimal average waiting time (SJF's goal) are different objectives, and no algorithm can be optimal at both simultaneously.

Q11. Scenario: You're designing a scheduler for a general-purpose desktop OS where you don't know in advance whether user programs will be interactive (e.g., a text editor) or CPU-heavy (e.g., video rendering). Which algorithm would you reach for, and why?

Answer: MLFQ. Since you can't know a process's nature in advance, you need a scheduler that classifies processes based on observed runtime behavior rather than requiring that information upfront (which rules out SJF/SRTF, and rules out a fixed MLQ classification made at process-creation time). MLFQ naturally keeps interactive processes (text editor, waiting on keystrokes) responsive in high-priority queues since they give up the CPU quickly, while CPU-heavy processes (video rendering) sink to lower-priority queues with larger time quanta where they can run efficiently without constantly interrupting interactive work — all without the scheduler needing to be told which is which ahead of time.